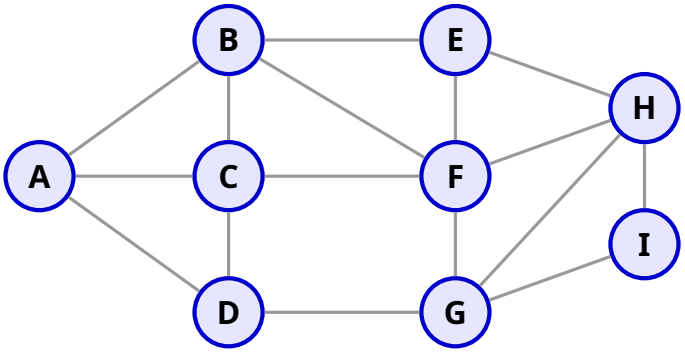
Graph Representation Practice

Data Structures

Problem 1: Undirected Graph Representation

The following undirected graph has nine nodes labeled **A** through **I**. Each line in the diagram represents an edge. Because the graph is undirected, every edge can be traversed in either direction.



1a. Adjacency Matrix

Fill each square with **1** if the node at the beginning of the row is adjacent to the node at the top of the column, and **0** otherwise. Use the same node order for rows and columns: **A, B, C, D, E, F, G, H, I**.

	A	B	C	D	E	F	G	H	I
A									
B									
C									
D									
E									
F									
G									
H									
I									

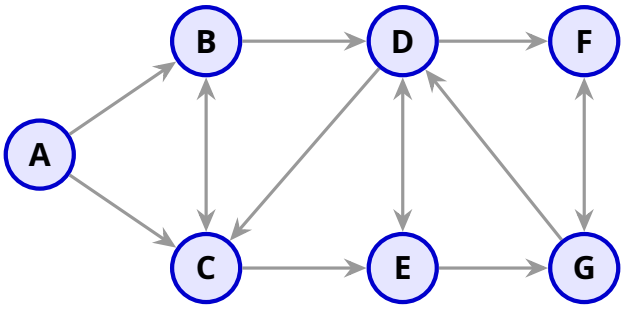
1b. Adjacency List

For each node, list all of its adjacent nodes. You may list neighbors in any order.

Node	Adjacent Nodes
A	
B	
C	
D	
E	
F	
G	
H	
I	

Problem 2: Directed Graph Representation

The following directed graph has seven nodes labeled **A** through **G**. Each arrow represents a directed edge pointing from a source node to a target node. Double-headed arrows represent bidirectional edges (traversable in both directions).



2a. Directed Adjacency Matrix

Fill each square with **1** if there is a directed edge pointing from the row node to the column node, and **0** otherwise.

	A	B	C	D	E	F	G
A							
B							
C							
D							
E							
F							
G							

2b. Directed Adjacency List

For each node, list all nodes that can be reached directly via an outgoing edge.

Node	Outgoing Neighbor Nodes
A	
B	
C	
D	
E	
F	
G	

Problem 3: Graph Traversal

A **graph traversal** is a systematic process of visiting (checking, processing, or updating) every vertex/node in a graph exactly once by following connected edges starting from a designated source node.

a. **Algorithm Design:** Design a step-by-step algorithm or procedure to perform a graph traversal starting from node **A**. How will your algorithm decide which node to visit next, and how will it ensure that no node is revisited or missed?

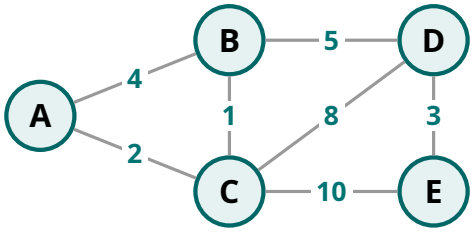
b. **Traversal Exercise:** Apply your traversal strategy to the undirected graph in **Problem 1**, starting at node **A**.

- Perform a **Breadth-First Search (BFS)** starting from **A**: _____
- Perform a **Depth-First Search (DFS)** starting from **A**: _____

Problem 4: More Graphs to Try

For each of the following graphs, construct its adjacency matrix and adjacency list representations, then trace both **BFS** and **DFS** traversals starting at the specified node.

4a. Weighted Undirected Graph

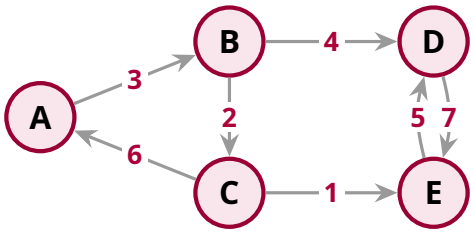


	A	B	C	D	E
A					
B					
C					
D					
E					

Node	Neighbors (Weight)
A	
B	
C	
D	
E	

- **BFS starting at Node C:** _____
- **DFS starting at Node C:** _____

4b. Weighted Directed Graph

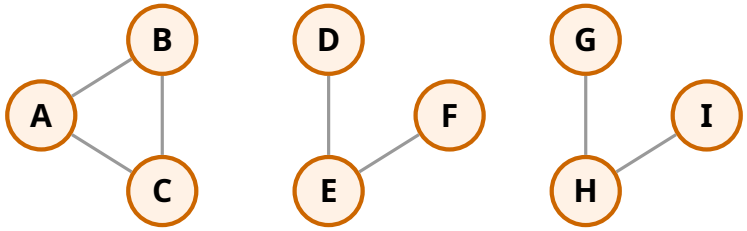


	A	B	C	D	E
A					
B					
C					
D					
E					

Node	Outgoing Neighbors (Weight)
A	
B	
C	
D	
E	

- **BFS starting at Node B:** _____
- **DFS starting at Node B:** _____

4c. Disconnected Unweighted Graph



	A	B	C	D	E	F	G	H	I
A									
B									
C									
D									
E									
F									
G									
H									
I									

Node	Adjacent Nodes
A	
B	
C	
D	
E	
F	
G	
H	
I	

- **BFS starting at Node E**: _____
- **DFS starting at Node E**: _____

Answer Sheet & Solutions Key

Problem 1 Solutions

1a. Adjacency Matrix

	A	B	C	D	E	F	G	H	I
A	0	1	1	1	0	0	0	0	0
B	1	0	1	0	1	1	0	0	0
C	1	1	0	1	0	1	0	0	0
D	1	0	1	0	0	0	1	0	0
E	0	1	0	0	0	1	0	1	0
F	0	1	1	0	1	0	1	1	0
G	0	0	0	1	0	1	0	1	1
H	0	0	0	0	1	1	1	0	1
I	0	0	0	0	0	0	1	1	0

1b. Adjacency List

Node	Adjacent Nodes
A	B, C, D
B	A, C, E, F
C	A, B, D, F
D	A, C, G
E	B, F, H
F	B, C, E, G, H
G	D, F, H, I
H	E, F, G, I
I	G, H

Problem 2 Solutions

2a. Directed Adjacency Matrix

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	0	0	1	1	0	0	0
C	0	1	0	0	1	0	0
D	0	0	1	0	1	1	0
E	0	0	0	1	0	0	1
F	0	0	0	0	0	0	1
G	0	0	0	1	0	1	0

2b. Directed Adjacency List

Node	Outgoing Neighbor Nodes
A	B, C
B	C, D
C	B, E
D	C, E, F
E	D, G
F	G
G	D, F

Problem 3 Solutions

- **Algorithm Design Idea**: Track visited nodes with a set or boolean array. Use a Queue for BFS (level-by-level) or a Stack / Recursion for DFS. Mark nodes as visited to avoid infinite cycles.
- **BFS starting at Node A** (Alphabetical order): **A, B, C, D, E, F, G, H, I**
- **DFS starting at Node A** (Alphabetical tie-breaking): **A, B, C, D, G, F, E, H, I**

Problem 4 Solutions

4a. Weighted Undirected Graph Solutions

Adjacency Matrix (Weights):

	A	B	C	D	E
A	0	4	2	0	0
B	4	0	1	5	0
C	2	1	0	8	10
D	0	5	8	0	3
E	0	0	10	3	0

Adjacency List:

Node	Neighbors (Weight)
A	B (4), C (2)
B	A (4), C (1), D (5)
C	A (2), B (1), D (8), E (10)
D	B (5), C (8), E (3)
E	C (10), D (3)

- **BFS starting at Node C**: **C, A, B, D, E**
- **DFS starting at Node C**: **C, A, B, D, E**

4b. Weighted Directed Graph Solutions

Directed Adjacency Matrix (Weights):

	A	B	C	D	E
A	0	3	0	0	0
B	0	0	2	4	0
C	6	0	0	0	1
D	0	0	0	0	7
E	0	0	0	5	0

Directed Adjacency List:

Node	Outgoing Neighbors (Weight)
A	B (3)
B	C (2), D (4)
C	A (6), E (1)
D	E (7)
E	D (5)

- **BFS starting at Node B**: **B, C, D, A, E**
- **DFS starting at Node B**: **B, C, A, E, D**

4c. Disconnected Unweighted Graph Solutions

Adjacency Matrix:

	A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
A	0	1	1	0	0	0	0	0	0
B	1	0	1	0	0	0	0	0	0
C	1	1	0	0	0	0	0	0	0
D	0	0	0	0	1	0	0	0	0
E	0	0	0	1	0	1	0	0	0
F	0	0	0	0	1	0	0	0	0
G	0	0	0	0	0	0	0	1	0
H	0	0	0	0	0	0	1	0	1
I	0	0	0	0	0	0	0	1	0

Adjacency List:

Node	**Adjacent Nodes**
A	B, C
B	A, C
C	A, B
D	E
E	D, F
F	E
G	H
H	G, I
I	H

- **BFS starting at Node E**: **E, D, F** (*Nodes in other components are unreachable*)
- **DFS starting at Node E**: **E, D, F** (*Nodes in other components are unreachable*)